



Lenguajes de Programación

Módulo II. Python

Tema 3: Conceptos básicos



©2026

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Introducción

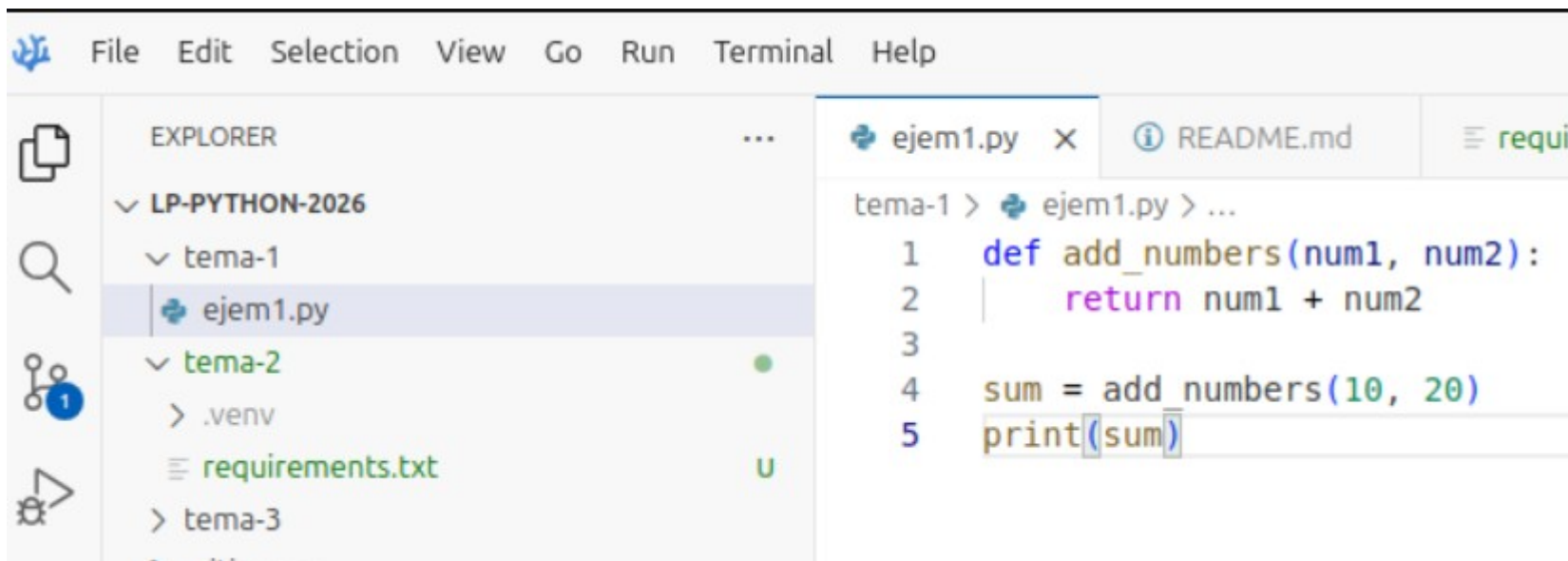


- Tipos de datos básicos
- Tipos estructurados
- Control de flujo

¿Dónde pongo mi código?



- El código en Python se escribe en ficheros con extensión “.py”



Tipos de datos básicos



- **Cadenas de caracteres o strings (clase str)**
 - Inmutables
 - Pueden delimitarse con comillas dobles o simples:
 - `"Hola, mundo"`
 - `'Hola, mundo'`
 - Pueden incluirse comillas de un tipo dentro de un string delimitado con el otro tipo:
 - `"Hola, 'mundo' "`
 - Pueden indexarse:
 - `"Hola, mundo"[0] == "H"`
 - `"Hola, mundo"[-1] == "o"`
 - Concatenación:
 - `"Hola" + ", " + "mundo" → "Hola, mundo"` (genera un nuevo string)
 - Si comienzan con tres delimitadores pueden expandirse a varias líneas:

```
"""Hola,  
mundo"""  
'''Hola,  
mundo'''
```

Tipos de datos básicos



- **Enteros (clase int)**
 - Representa números enteros positivos, negativos y el cero
 - Tienen longitud arbitraria
 - Ejemplos:
 - 15, -6, 123946404782147032471
 - Conversión a entero con la función `int()`:
 - `int("16")`

Tipos de datos básicos



- **Coma flotante o decimales (clase float)**
 - Representa números con decimales
 - Dos notaciones:
 - Notación punto: 4.8, 0.5, .5
 - Notación científica: 3.7e-8
 - Conversión con float():
 - `float("12.6")` → 12.6
 - `float(4)` → 4.0
 - El operador de división siempre da como resultado un número decimal:
 - `4 / 2` → 2.0

Tipos de datos básicos



- **Lógicos o booleanos (clase bool)**
 - Representan los valores verdadero o falso
 - Se escriben con la primera letra en mayúsculas:
 - **True**
 - **False**
 - Operadores booleanos:
 - **and, or, not**

Entrada/Salida



- **Salida**

- Función `print()`

- `print("Hola, mundo")`

- **Entrada**

- Función `input()`

- `nombre = input("Dime un nombre: ")`

Tipos de datos estructurados



- **Listas**

- Objetos de la clase list (librería estándar)
- Secuencias ordenadas y mutables de elementos
- Pueden contener elementos de diferentes tipos
- Creación:
 - Lista vacía: `[]`
 - Lista inicializada con valores: `users = ['alex', 'mia', 'peter']`
 - Lista con diferentes tipos de datos: `chaos = ['dog', 45, True]`

Tipos de datos estructurados



- **Listas**

- Añadir al final:

- `[ 2, 3, 4 ].append(5) → [ 2, 3, 4, 5 ]`

- Insertar en una posición desplazando:

- `[ 2, 3, 4 ].insert(1, 5) → [ 2, 5, 3, 4 ]`

Tipos de datos estructurados



- **Listas**

- Eliminar:
 - `Lista = [ 2, 3, 4 ]`
 - `del lista[1] → lista == [ 2, 4 ]`
- Eliminar la primera aparición de un valor:
 - `[ 2, 3, 4, 3 ].remove(3) → [ 2, 4, 3 ]`
- Eliminar y devolver el último elemento:
 - `[ 2, 3, 4 ].pop() → [ 2, 3 ]`
- O el primero:
 - `[ 2, 3, 4 ].pop(0) → [ 3, 4 ]`

Tipos de datos estructurados



- **Listas**

- Acceso:

- Los índices comienzan en 0

- `lista = [ 2, 3, 4 ]`

- `lista[1] == 3`

- Acceso por el final (índices negativos):

- `lista[-1] == 4`

- Modificación:

- `lista[1] = 5 → [ 2, 5, 4 ]`

Tipos de datos estructurados



- **Listas**

- Ordenación:

- `Lista = [ 5, 4, 3 ]`

- Ordenar sobre la propia lista:

- `lista.sort() → lista == [ 3, 4, 5 ]`

- Devolver una copia ordenada sin modificar la original:

- `sorted(lista) → [3, 4, 5] && lista == [ 5, 4, 3 ]`

- Revertir una lista

- `[ 5, 4, 3 ].reverse() → [ 3, 4, 5 ]`

Tipos de datos estructurados



- **Listas**

- Slicing (segmentación):
 - Sintaxis: [inicio:fin] o [:fin] o [inicio:] o [:]
 - Devuelve una nueva lista (la original no es modificada)
 - Útil para pasar listas a funciones cuando no queremos que se modifique la original
 - Ejemplo 2

Tipos de datos estructurados



- **Listas**
 - List comprehensions (listas por comprensión):
 - Se crea una lista a partir de una función generadora de valores
 - `cuadrados = [ x**2 for x in range(1, 11)]`

Tipos de datos estructurados



- **Diccionarios**
 - Pares clave, valor
 - Mutables
 - Sin orden, pero de rápido acceso (con la clave)
 - La clave debe ser de un tipo de datos inmutable (cadenas, números o tuplas)

Tipos de datos estructurados



- **Diccionarios**

- Creación:

- `ids = {}`
 - `ids = { 'peter' : 1, 'mia' : 2 }`

- Acceso:

- `ids['peter'] → 1`
 - `ids['john'] → error`
 - `ids.get('john') → None`

Tipos de datos estructurados



- **Diccionarios**
 - Añadir claves nuevas:
 - `ids['john'] = 5`
 - Modificar claves existentes:
 - `ids['peter'] = 10`
 - Eliminar un par clave, valor:
 - `del ids['peter']`

Tipos de datos estructurados



- **Diccionarios**

- Iteración:

- `ids.items()` → `[('mia', 2), ('john', 5)]`

- `ids.keys()` → `['mia', 'john']`

- `ids.values()` → `[2, 5]`

- Ejemplo 3

Tipos de datos estructurados



- **Tuplas**
 - Secuencia ordenada e inmutable de elementos de cualquier tipo
 - Una vez creado un valor tupla no se puede modificar

Tipos de datos estructurados



- **Tuplas**
 - Creación:
 - `vacía = ()`
 - `un_elemento = (1, )`
 - `varios_elementos = (1, 2, True, 'dog')`

Tipos de datos estructurados



- **Tuplas**
 - Acceso:
 - `varios_elementos[0] → 1`
 - `varios_elementos[-1] → 'dog'`

Tipos de datos estructurados



- **Sets**
 - Colección no ordenada de elementos sin duplicados
 - Mutables
 - No son indexables
 - Permiten operaciones como unión, intersección, diferencia...
 - Todos sus elementos deben ser inmutables

Tipos de datos estructurados



- **Sets**

- Creación:

- Conjunto vacío:

- `mi_set = set()`

- Conjunto con valores:

- `mi_set = { 1, 2, 3 }`

- Por comprensión:

- `otro_set = { n*n for n in range(5) }`

Tipos de datos estructurados



- **Sets**

- Añadir elementos:

- `mi_set.add(4) → { 1, 2, 3, 4 }`

- Eliminar elementos:

- `mi_set.remove(2) → { 1, 3, 4 }`

- `mi_set.discard(1) → { 3, 4 }`

Tipos de datos estructurados



- **Sets**
 - Comprobación de membresía:
 - `4 in mi_set` → `True`
 - `5 in mi_set` → `False`

Tipos de datos estructurados



- **Sets**

- $a = \{ 1, 2, 3, 4 \}$
- $b = \{ 3, 4, 5, 6 \}$
- Unión:
 - $a \mid b \rightarrow \{ 1, 2, 3, 4, 5, 6 \}$
- Intersección:
 - $a \& b \rightarrow \{ 3, 4 \}$
- Diferencia:
 - $a - b \rightarrow \{ 1, 2 \}$
- No comunes:
 - $a \wedge b \rightarrow \{ 1, 2, 5, 6 \}$

Control de flujo



- If
 - Sintaxis:
 - if <comparación>: # (ojo los dos puntos)
 - # cuerpo del if (ojo indentación)
 - elif <comparación>:
 - # otro cuerpo
 - elif <comparación>:
 - # otro más
 - else:
 - # aquí viene el else

Control de flujo



- **If ternario**

- Sintaxis:

- `x = 10 if <comparación>`

- Ejemplo:

- `odd = True if num % 2 == 0`

Control de flujo



- **While**

- Sintaxis:

```
while <comparación>: # (ojo los dos puntos)  
    # cuerpo del while
```

Control de flujo



- **For**

- Sintaxis:

`for name in names: # (ojo los dos puntos)`

`# cuerpo del for (ojo indentación)`

`# esto ya no es el for`

Control de flujo



- **For**
 - Convención:
 - La variable que almacena el elemento de la interacción actual se nombra en singular
 - La colección sobre la que se itera, en plural
- for **name** in **names**:

Control de flujo



- **For**

- Para hacer bucles que se repiten un número determinado de veces:

```
for i in range(100):  
    print i
```

Control de flujo



- **For**

- Para recorrer listas:

```
names = ['mia', 'peter', 'john']
```

```
for name in names:
```

```
    print(name)
```

Control de flujo



- **For**

- Para recorrer diccionarios:

```
ids = ['mia':1, 'peter':2, 'john':3 ]
```

```
for name, id in ids.items():
```

```
    print(name + ":" + id)
```

Control de flujo



- **For**

- Para recorrer diccionarios:

```
ids = ['mia':1, 'peter':2, 'john':3 ]
```

```
for id in ids.values():
```

```
    print(id)
```

Control de flujo



- **For**

- Para recorrer diccionarios:

```
ids = ['mia':1, 'peter':2, 'john':3 ]
```

```
for name in ids.keys():
```

```
    print(name)
```

Control de flujo



- **For**
 - También podemos recorrer sets o strings
 - Se puede alterar el comportamiento del bucle con:
 - **break**: sale inmediatamente del bloque
 - **continue**: salta inmediatamente al comienzo de la siguiente iteración

Control de flujo



- **Match (desde Python 3.10)**
 - Permite seleccionar utilizando pattern matching (coincidencia de patrones)

```
match status:
    case 400:
        return "Bad request"
    case 401 | 403 | 404: # Agrupación de valores
        return "Not allowed"
    case 404:
        return "Not found"
    case 418:
        return "I'm a teapot"
    case _:
        return "Something's wrong with the internet"
```


Control de flujo



- **Match (desde Python 3.10)**
- Es más interesante cuando se utiliza el pattern matching con deconstrucción de objetos:

```
point=(5, 0)
match point:
    case (0, 0):
        print("Origin")
    case (0, y):
        print(f"Y={y}")
    case (x, 0):
        print(f"X={x}")
    case (x, y):
        print(f"X={x}, Y={y}")
    case _:
        raise ValueError("Not a point")
```

Control de flujo



- **Match (desde Python 3.10)**
- Es más interesante cuando se utiliza el pattern matching con deconstrucción de objetos:

```
point=(5, 0)
match point:
    case (0, 0):
        print("Origin")
    case (0, y):
        print(f"Y={y}")
    case (x, 0):
        print(f"X={x}")
    case (x, y):
        print(f"X={x}, Y={y}")
    case _:
        raise ValueError("Not a point")
```

Control de flujo



- Manejo de excepciones

- Sintaxis:

- try:

- # Bloque que puede lanzar error

- except FileNotFoundError:

- # Bloque para manejar el error

- except RuntimeError:

- # Otro bloque para otro error diferente

- else:

- # Bloque que se ejecuta sólo si no hay error

- finally:

- # Bloque que se ejecuta siempre

Control de flujo



- Manejo de excepciones
 - Ejemplo (manejo de ficheros)

```
try:
```

```
    f = open('myfile', 'r')
```

```
except OSError:
```

```
    print("Can't open file")
```

```
else:
```

```
    # Cerramos el fichero
```

```
    f.close()
```

Control de flujo



- **Operador with**
 - Cierra los recursos cuando se sale del bloque with
 - Ejemplo (manejo de ficheros)

`with open('myfile', 'r') as f:`

`# Imprimir el contenido del fichero`

`for line in f:`

`print(line)`